

High-Level Design

Project: Aura Promptly
Version: 1.1
Date: 2026-05-14
Status: Baselined (updated post-review)

Change Log

Version	Date	Changes
1.0	2026-05-14	Initial baseline
1.1	2026-05-14	GCP auth revised (direct OIDC→STS, no Cognito); vector store changed to Aurora pgvector; observability updated to two-layer; LiteLLM image pinned to -stable ; public repo security section added; Agent demo redefined as Research Assistant; KB sync poll-and-wait added to recreate runbook
1.2	2026-05-14	Added comprehensive draw.io diagrams (7 diagrams covering all design segments)

Diagrams Index

All diagrams are in [docs/diagrams/](#). Open with [draw.io \(diagrams.net\)](#) or the VS Code Draw.io Integration extension.

#	File	Scope	Key Content
1	01_overall_architecture.drawio	Complete PoC — Master View	All 5 hub layers, 3 spokes, auth labels on connections, legend, CI/CD tooling
2	02_hub_components.drawio	Hub Components Detail	Every component with specs (CPU, RAM, cost), internal data flows, 5-layer breakdown
3	03_network_design.drawio	Network Topology	Hub VPC (public/private subnets), 7 VPC endpoints, NAT Gateway, ALB, Security Groups, spoke connectivity
4	04_auth_flows.drawio	Authentication Flows	Step-by-step flows for all 3 auth patterns + sequence diagrams + auth decision matrix
5	05_data_flows.drawio	Data Flows	RAG chatbot flow (with KB sync), Research Assistant agent loop, Summarisation with model routing, guardrails filter chain

#	File	Scope	Key Content
6	06_observability.drawio	Observability Stack	Two-layer architecture (CloudWatch + OTel/Grafana), 4 dashboard specifications, LiteLLM spend dashboard, telemetry sources
7	07_cicd_pipeline.drawio	CI/CD Pipelines	Phase 1b (basic) and Phase 11 (full) pipelines, destroy pipeline, recreate runbook, deployment dependency graph

Tip for LinkedIn/YouTube content: Diagrams 1 and 5 are the most visually compelling for demo screenshots. Diagram 4 (auth flows) is good for explaining the zero-trust architecture to a technical audience.

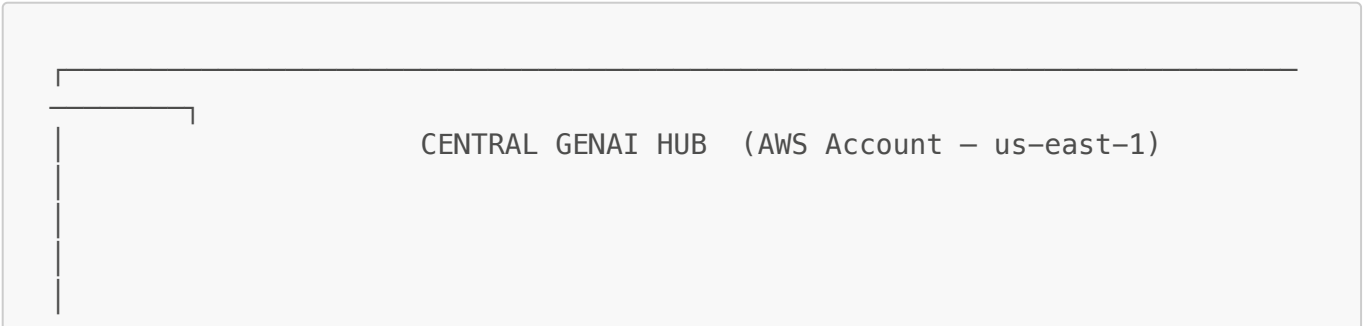
1. Architecture Overview

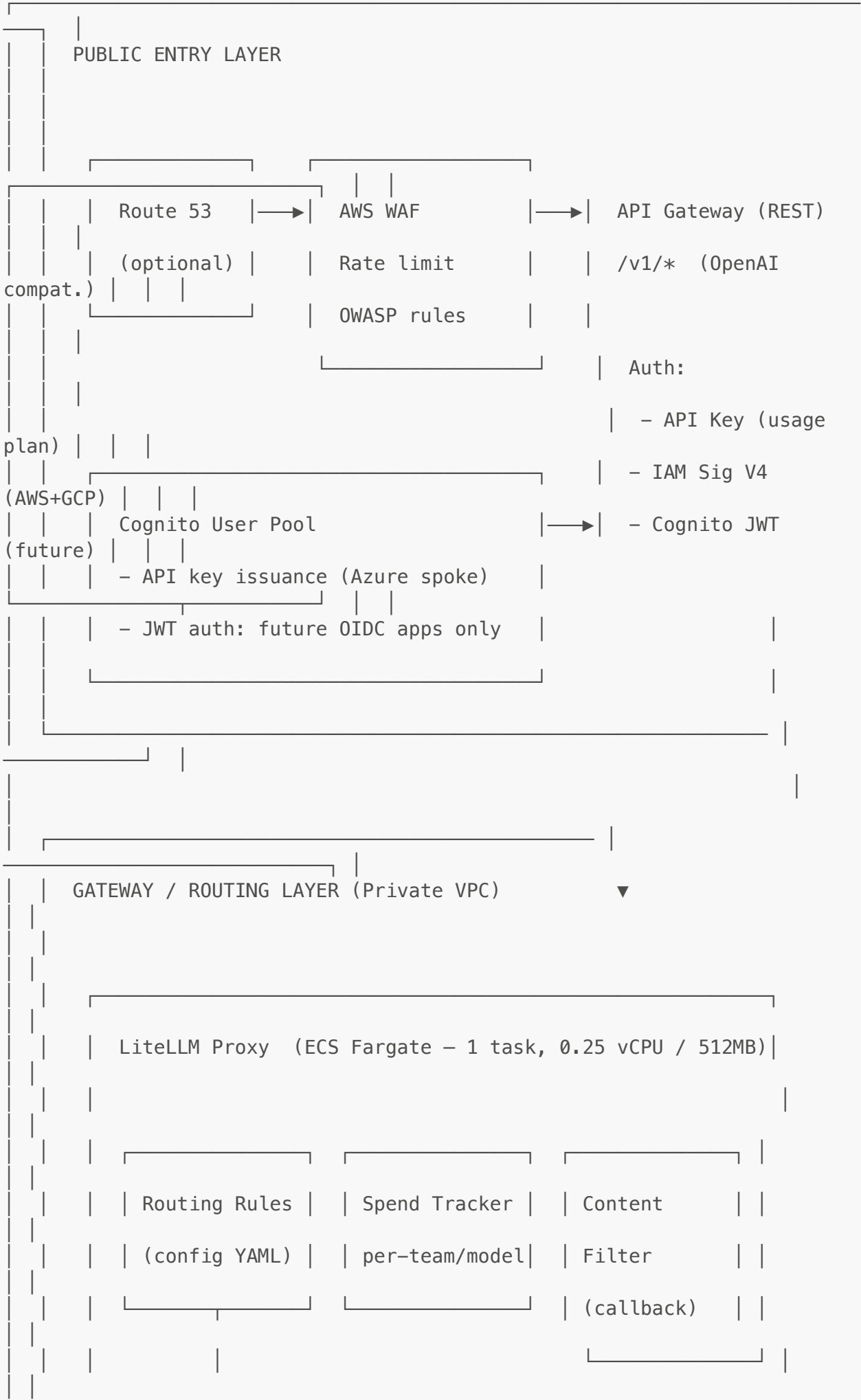
1.1 Design Principles

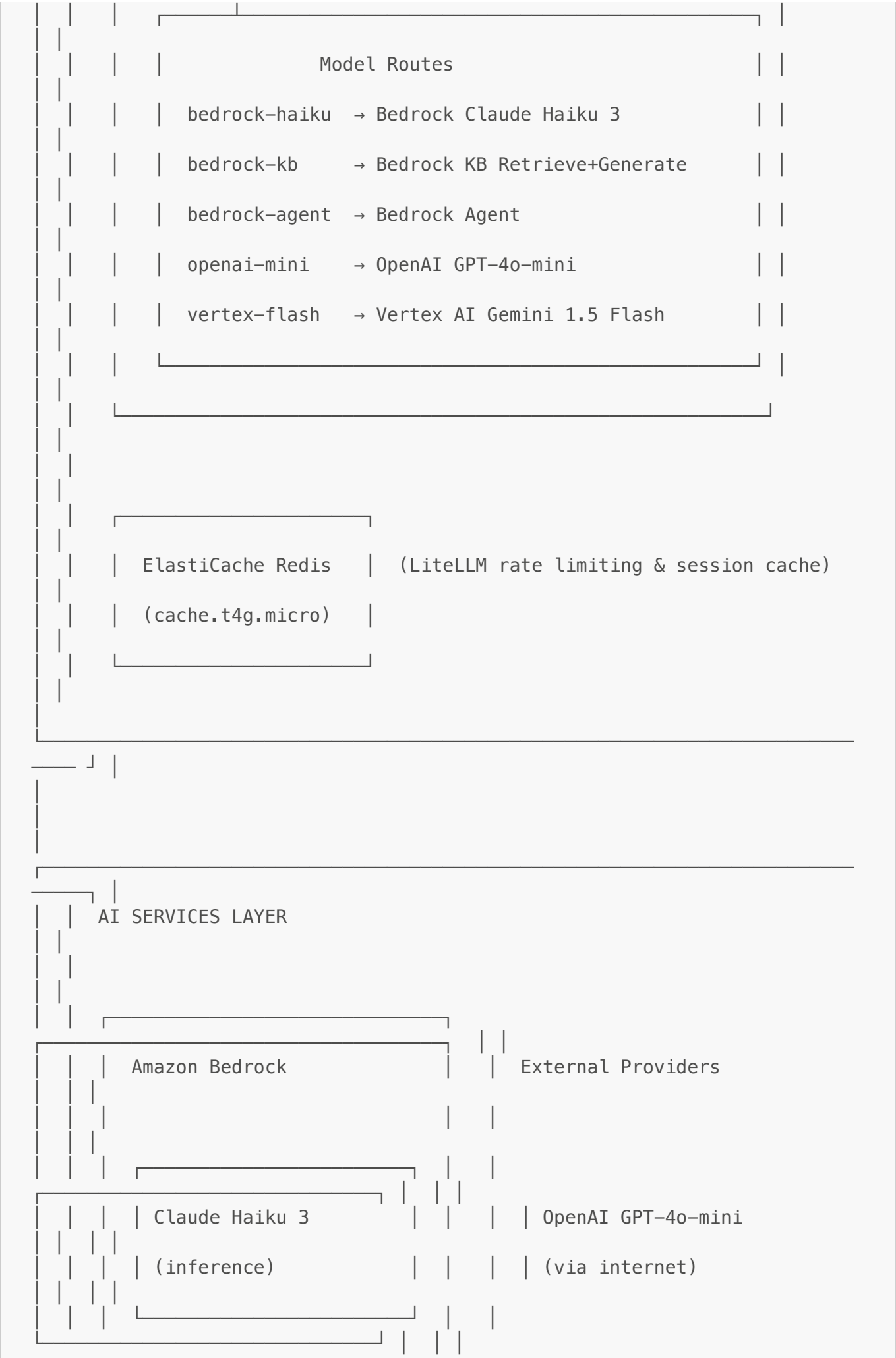
Principle	Application
Centralised governance, decentralised execution	AI control plane lives in the hub; applications run in spoke accounts
Zero trust network	All connections authenticated and encrypted; no implicit trust based on network location
Destroy-safe infrastructure	Every component can be torn down and recreated without data loss (except KB documents, which are in S3)
Cost first	On-demand pricing everywhere; serverless preferred; auto-scaling disabled in PoC
Well-Architected modularity	Each pillar (security, reliability, cost, ops, performance) is a toggleable concern in Terraform
Open standards	OpenTelemetry for observability; OpenAI-compatible API for portability

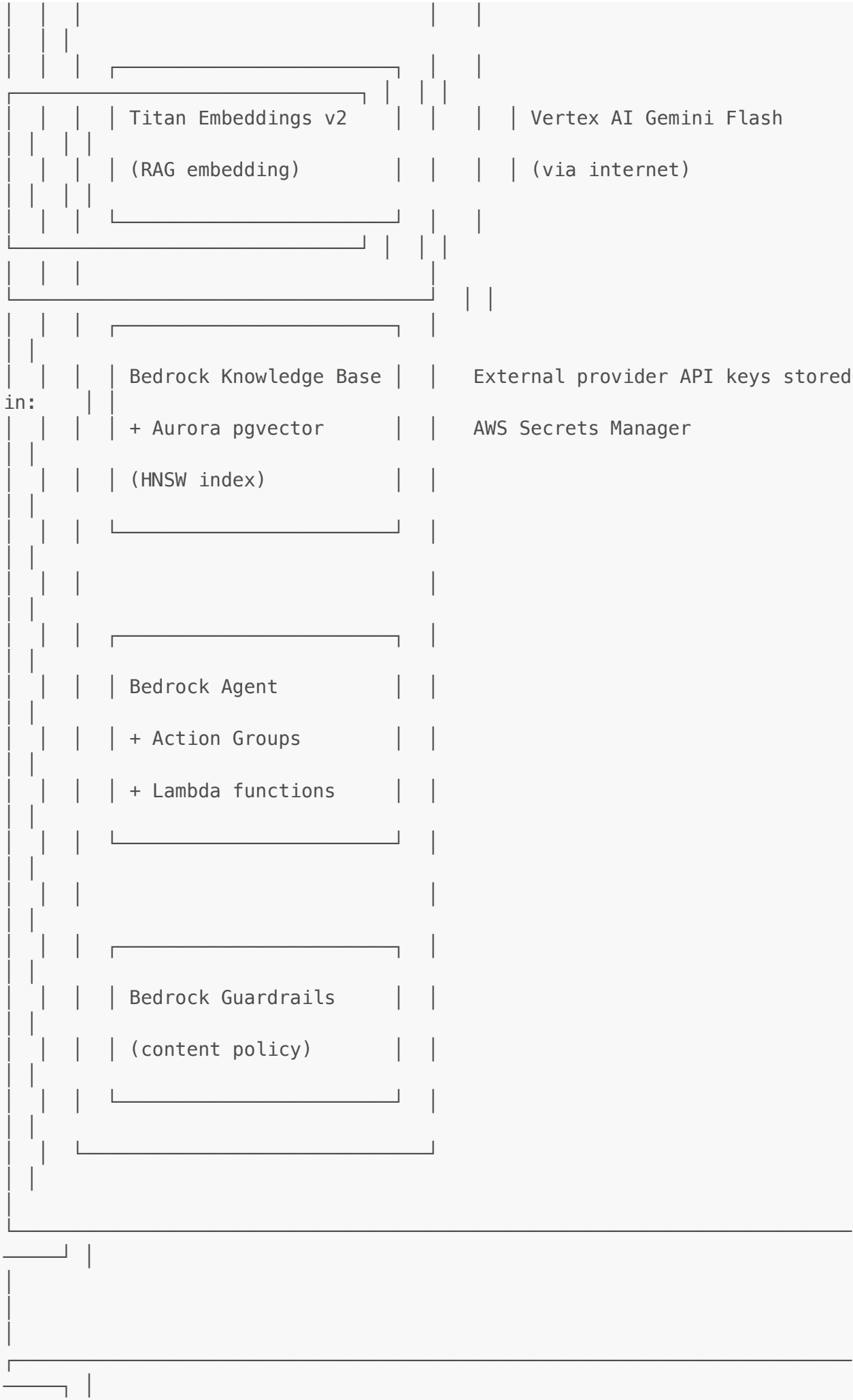
1.2 Logical Architecture

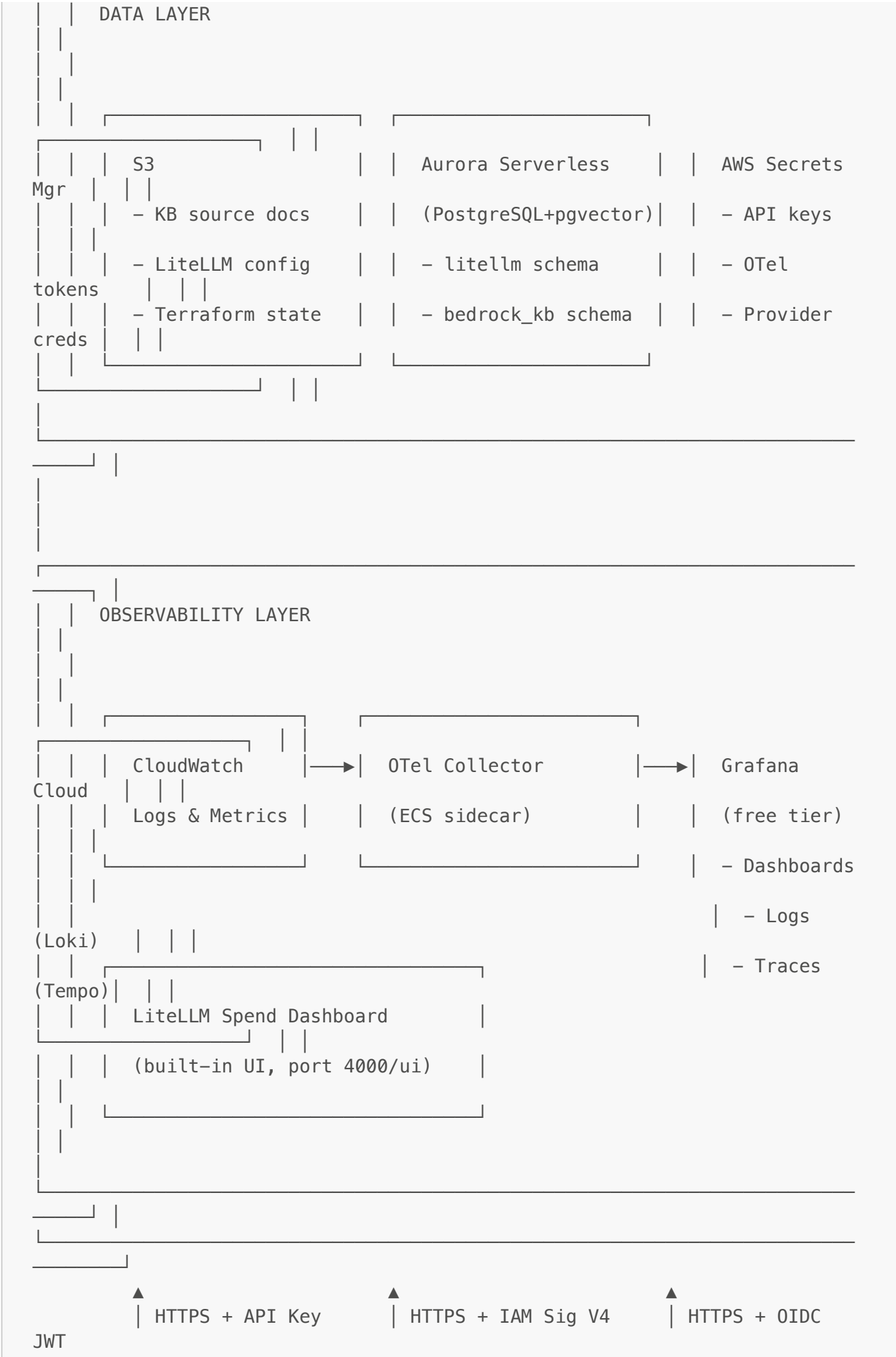
Diagram: [01_overall_architecture.drawio](#) — master view of all layers and spokes
Diagram: [02_hub_components.drawio](#) — hub detail with component specs and costs

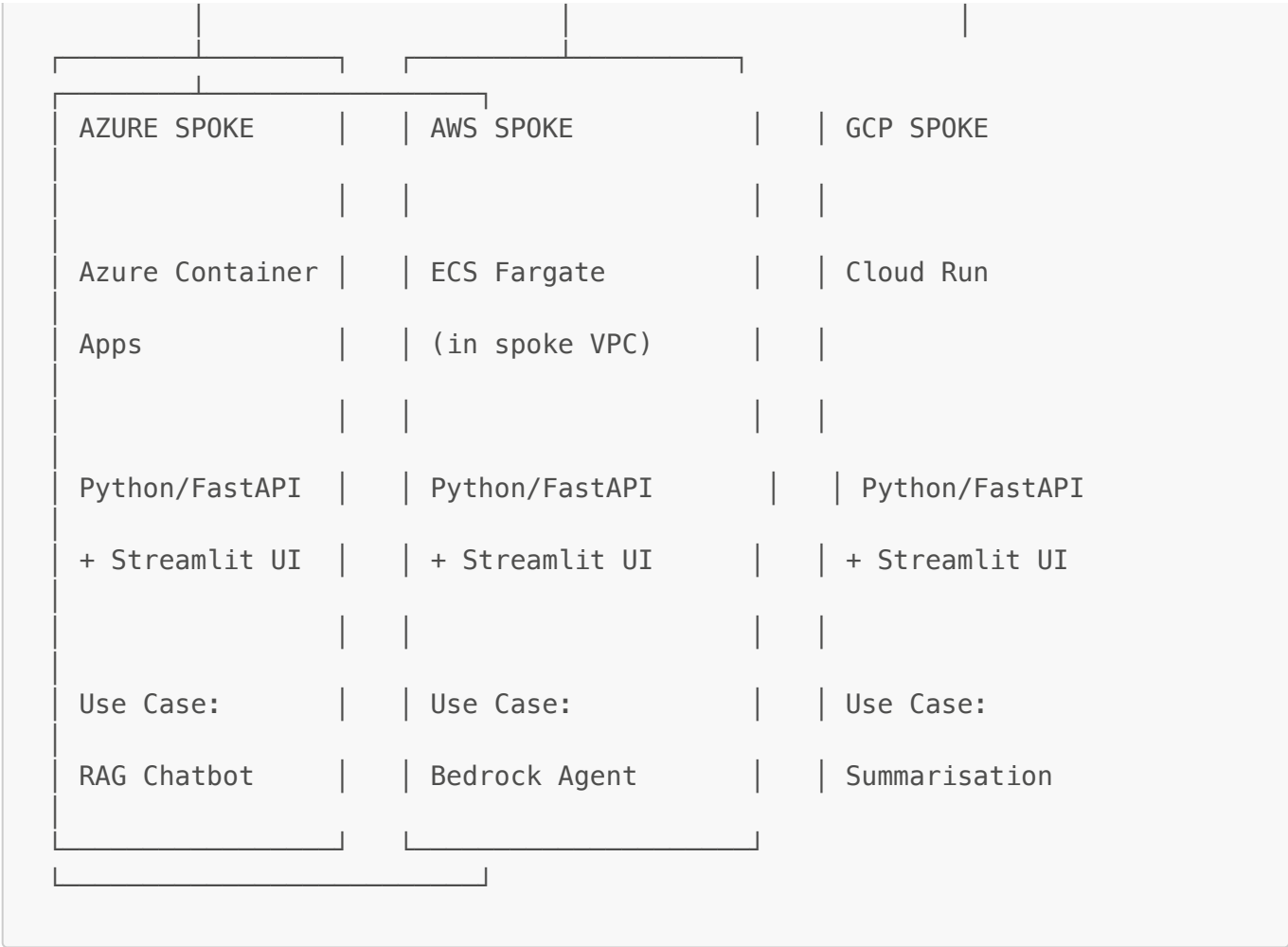












2. Component Descriptions

2.1 AWS API Gateway (Entry Point)

- **Type:** REST API with regional endpoint
- **Auth methods:**
 - API Key + Usage Plans (for Azure spoke — simple, cost-free)
 - AWS IAM Signature V4 (for AWS and GCP spokes — cross-account role and Workload Identity)
 - Cognito JWT Authoriser (future OIDC apps — not used by any current spoke)
- **Rate limiting:** WAF rule — 100 requests per 5 minutes per IP (PoC setting)
- **TLS:** Minimum TLS 1.2 enforced
- **Routes:** All traffic proxied to LiteLLM on `/v1/*` (OpenAI-compatible)
- **Cost:** ~\$3.50/million API calls (effectively ~\$0 for PoC volume)

2.2 AWS WAF

- **Managed rules:** AWS-AWSManagedRulesCommonRuleSet (OWASP Top 10)
- **Custom rules:** Rate-based rule (100 req/5min per IP), Geo-block (optional, disabled by default)
- **Cost:** \$5/month WAF WebACL + \$1/million requests (dominant cost in PoC — see cost doc)
- **Toggle:** Terraform variable `enable_waf = true/false` (disable to save \$5/month during dev)

2.3 LiteLLM Proxy (ECS Fargate)

- **Image:** `ghcr.io/berriai/litellm:v1.83.0-stable` (pinned to specific `-stable` semver; `main-latest` must NOT be used — see ADR-11)
- **Compute:** 0.25 vCPU, 512 MB RAM (1 task). Scales to 0 when destroyed.
- **Database:** Aurora Serverless PostgreSQL (for spend tracking, user management, API keys; shared cluster with KB pgvector schema)
- **Cache:** ElastiCache Redis (t4g.micro) for rate limiting and model response caching
- **Config:** YAML config mounted from S3 (hot-reload enabled); all credentials referenced via `os.environ/SECRET_NAME` — no literal values in YAML
- **Admin UI:** Exposed on port 4000/ui, accessible via separate ALB listener (not public)
- **Deployment:** Blue/green via ECS rolling update; cosign image signature verified in CI/CD before deploy
- **Cost:** ~\$0.01/hour for Fargate task when running

2.4 Amazon Bedrock

Model	Use	Cost (on-demand)
Claude Haiku 3 (anthropic.claude-3-haiku-20240307-v1:0)	Primary inference	\$0.00025/1K input tokens, \$0.00125/1K output
Titan Text Embeddings v2	RAG embedding generation	\$0.00002/1K tokens
Bedrock Guardrails	Content policy enforcement	\$0.75/1K text units

Region: us-east-1 (broadest model availability, lowest Haiku pricing)

Bedrock access request: Must be manually enabled in AWS Console → Bedrock → Model Access before Terraform apply.

2.5 Bedrock Knowledge Base

- **Data source:** S3 bucket (HTML/PDF/TXT documents)
- **Vector store:** Aurora Serverless PostgreSQL with pgvector extension (GA as Bedrock KB vector store, May 2026)
- **Embedding model:** Titan Text Embeddings v2 (1024 dimensions)
- **Chunking strategy:** Fixed size (512 tokens, 15% overlap) — configurable
- **Index type:** HNSW (pgvector) — pre-created by Terraform before KB registration
- **Sync trigger:** Manual (GitHub Actions workflow dispatch or AWS CLI); recreate runbook polls for `COMPLETE` status before deploying spokes
- **Aurora cluster:** Shared with LiteLLM DB (separate schema `bedrock_kb`) — no additional cluster cost
- **Future connectors (stubs):** Confluence, SharePoint, Salesforce

Cost note: Aurora Serverless scales to 0 ACUs when idle — effectively \$0 between sessions. This replaces OpenSearch Serverless (~\$172/month minimum) from v1.0. See ADR-05.

2.6 Bedrock Agent

- **Foundation model:** Claude Haiku 3
- **Demo scenario:** "Research Assistant" — user asks a governance question; Agent queries the Knowledge Base and synthesises a structured board-ready summary
- **Action Groups:** 1 Lambda function ([search_knowledge_base](#)) — retrieves chunks from Bedrock KB using the [retrieve](#) API
- **Session handling:** 30-minute session timeout
- **Access:** Via LiteLLM native route [bedrock/agent/{AGENT_ID}/{ALIAS_ID}](#) — no custom Lambda adapter required (resolves OQ-04)

Lambda Action Group — [search_knowledge_base](#):

```
def handler(event, context):
    kb_id = os.environ["KNOWLEDGE_BASE_ID"]
    query = event["inputText"]
    response = bedrock_agent_runtime.retrieve(
        knowledgeBaseId=kb_id,
        retrievalQuery={"text": query},
        retrievalConfiguration={"vectorSearchConfiguration":
{"numberOfResults": 5}}
    )
    return {"results": [r["content"]["text"] for r in
response["retrievalResults"]]}
```

2.7 Amazon Cognito

- **User Pool:** Hosts M2M client credentials for API key issuance (Azure spoke)
- **Note:** Cognito is no longer used for GCP OIDC federation — GCP now uses direct AWS STS federation (see ADR-09 and Section 4.1)
- **Cost:** Free tier covers PoC volume (50k MAUs free)

2.8 Observability Stack

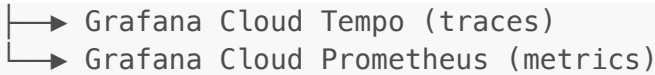
Diagram: [06_observability.drawio](#) — full two-layer architecture, telemetry sources, 4 dashboard specs, LiteLLM spend UI

Two-layer architecture:

```

LiteLLM (stdout JSON logs)
├─ CloudWatch Logs (log group: /aura-promptly/litellm) ← LAYER 1:
Always-on, no egress
├─ CloudWatch Metrics (EMF embedded metric format)
├─ CloudWatch Dashboards (4 dashboards – same content as
Grafana)
├─ CloudWatch Alarms (error rate, P95 latency, spend
threshold)
└─ OTel Collector (ECS sidecar) ← LAYER 2:
Optional, enable_grafana_export=true
    └─ Grafana Cloud Loki (logs)

```



Layer 1 — CloudWatch (always-on):

- Zero data egress — all logs stay within AWS
- Suitable for regulated/financial services environments
- CloudWatch Dashboards provide the same 4 views as Grafana
- CloudWatch Alarms send email on error rate spike or spend threshold breach

Layer 2 — Grafana Cloud (optional, `enable_grafana_export = true`):

- Grafana Cloud free tier: 10k active metrics series, 50GB logs/month, 14-day retention
- Visually compelling dashboards for YouTube/LinkedIn demo content
- OTEL pipeline is vendor-neutral — same config targets Datadog, Splunk, or Azure Monitor by changing the exporter endpoint
- Grafana API token stored in Secrets Manager; never committed to repository

Dashboards (both layers):

1. **Platform Overview:** Total requests, error rate, P95 latency
2. **Spend per Team:** Bar chart of token costs by team per day
3. **Model Usage:** Pie chart of requests by model
4. **Guardrail Events:** Table of blocked requests with reason

2.9 Data Layer

Store	Purpose	Destroy-safe?
S3 (KB bucket)	Knowledge Base source documents	Yes — documents re-uploaded via runbook
S3 (config)	LiteLLM config YAML	Yes — in git
Aurora Serverless	LiteLLM spend/user data (schema: <code>litellm</code>) + KB pgvector (schema: <code>bedrock_kb</code>)	Yes — data is ephemeral for PoC
Secrets Manager	API keys and credentials	Yes — recreated by Terraform
ElastiCache Redis	Rate limit state, response cache	Yes — ephemeral

3. Network Design

Diagram: [03_network_design.drawio](#) — VPC topology, subnets, VPC endpoints, Security Groups, spoke connectivity

3.1 Hub VPC



```
Hub VPC: 10.0.0.0/16

Public Subnets (10.0.1.0/24, 10.0.2.0/24):
- NAT Gateway (for Fargate tasks to reach internet / Bedrock)
- Application Load Balancer (for LiteLLM, internal only)

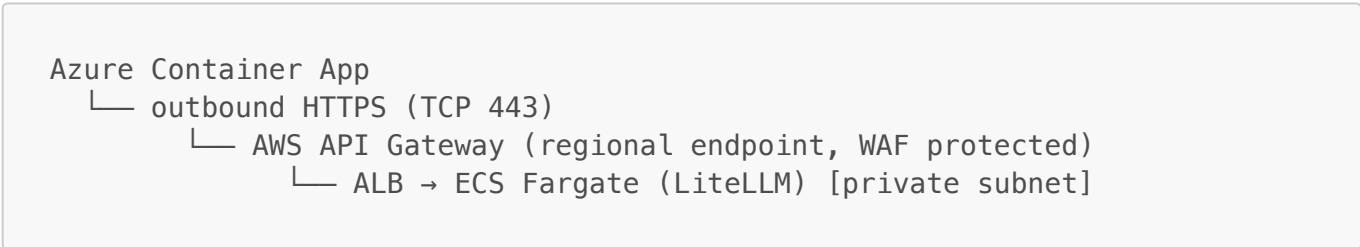
Private Subnets (10.0.10.0/24, 10.0.11.0/24):
- ECS Fargate tasks (LiteLLM + OTel Collector)
- Aurora Serverless
- ElastiCache Redis

VPC Endpoints (to avoid NAT costs):
- com.amazonaws.us-east-1.bedrock-runtime
- com.amazonaws.us-east-1.bedrock-agent-runtime
- com.amazonaws.us-east-1.secretsmanager
- com.amazonaws.us-east-1.s3 (gateway endpoint - free)
- com.amazonaws.us-east-1.ecr.api
- com.amazonaws.us-east-1.ecr.dkr
- com.amazonaws.us-east-1.logs
```

Note: VPC endpoints for Bedrock save NAT Gateway data processing charges for Bedrock API calls.

3.2 Spoke Connectivity

All spokes connect to the hub over the public internet via HTTPS to AWS API Gateway. No VPN or private peering required.



3.3 Spoke Network Footprint (Minimal)

Spoke	Compute	Network
Azure	Azure Container Apps (serverless, no VNet required)	Outbound internet only
AWS	ECS Fargate in spoke VPC, or Lambda	Outbound via NAT Gateway or Lambda -> internet
GCP	Cloud Run (serverless, no VPC required)	Outbound internet only

4. Authentication & Authorisation

Diagram: [04_auth_flows.drawio](#) — step-by-step flows + sequence diagrams for all 3 auth patterns

4.1 Auth Decision Matrix

Spoke	Auth Method	Why
Azure	API Key (x-api-key header)	Simplest, no AWS footprint needed, key stored in Azure Key Vault
AWS	IAM Signature V4 (cross-account STS role)	Native AWS auth, no long-lived credentials
GCP	IAM Signature V4 via GCP Workload Identity OIDC → AWS STS	Direct federation, no Cognito, lower latency, AWS-documented pattern

4.2 Cross-Account IAM (AWS Spoke)

AWS Spoke Account

- └ IAM Role: aura-promptly-spoke-role
 - └ Trust policy: allows hub account to be called via STS
 - └ Permissions: execute-api:Invoke on hub API Gateway ARN

Hub Account

- └ API Gateway IAM authoriser validates the assumed role ARN

4.3 GCP Workload Identity → AWS STS (GCP Spoke)

GCP Cloud Run service account

- └ GCP metadata server issues OIDC identity token
 - └ AWS STS AssumeRoleWithWebIdentity
 - └ Temporary AWS credentials (15-min TTL, auto-refreshed)
 - └ AWS Signature V4 call to Hub API Gateway
 - └ API Gateway IAM authoriser validates role

ARN

IAM trust policy (hub account):

```
{
  "Effect": "Allow",
  "Principal": { "Federated": "accounts.google.com" },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "accounts.google.com:sub": "<GCP_SERVICE_ACCOUNT_UNIQUE_ID>"
    }
  }
}
```

Benefits over original Cognito approach:

- One fewer token exchange step (~50ms latency saving)
- Single IAM authoriser on API Gateway handles both AWS and GCP spokes

- AWS-documented pattern (AWS Security Blog, Dec 2023)
- No Cognito dependency for GCP

4.4 API Key Lifecycle (Azure Spoke)

1. Platform engineer creates team in LiteLLM Admin UI (or Terraform)
2. LiteLLM generates API key, stores hash in Aurora DB
3. API Gateway usage plan linked to key (rate limit enforced at gateway)
4. Team stores key in Azure Key Vault – never in code or environment variables
5. Key passed as x-api-key header on every request
6. LiteLLM validates key against DB, checks team budget

5. Guardrails Architecture

Diagram: [05_data_flows.drawio](#) — guardrails filter chain section at the bottom of the data flows diagram

Request flow with guardrails:

```

App → API GW → LiteLLM → [LiteLLM Content Filter] → Bedrock
Guardrails → Bedrock Model
                                (pre-call callback)      (applied
at                               - keyword deny list      model
invocation)                  - prompt injection detect

```

If blocked at LiteLLM: HTTP 400 {"error": "content_policy_violation",
"source": "litellm"}

If blocked at Bedrock: HTTP 400 {"error": "content_policy_violation",
"source": "bedrock_guardrail"}

If allowed: Normal response

Bedrock Guardrails configuration (PoC defaults):

- Hate speech: BLOCK
- Sexual content: BLOCK
- Violence: BLOCK
- Insults: BLOCK
- Misconduct: BLOCK
- Denied topics: List TBD (financial advice, medical advice as examples)
- Word filters: Configurable list

6. CI/CD Pipeline Design

Diagram: [07_cicd_pipeline.drawio](#) — Phase 1b, Phase 11, destroy pipeline, recreate runbook, dependency graph

Phase 1b — Basic Pipeline (from Phase 1 onwards):

```

GitHub Repository (main branch protected)
├── Pull Request → GitHub Actions: Basic PR Pipeline
│   ├── gitleaks secret scan
│   ├── terraform fmt -check
│   ├── terraform validate
│   └── terraform plan (hub workspace)
└── Merge to main → GitHub Actions: Basic Deploy Pipeline
    └── terraform apply (hub)
  
```

Phase 11 — Full Pipeline:

```

GitHub Repository (main branch protected)
├── Pull Request → GitHub Actions: Full PR Pipeline
│   ├── gitleaks secret scan
│   ├── terraform fmt -check
│   ├── terraform validate
│   ├── terraform plan (all workspaces)
│   ├── infracost diff (posts cost delta to PR comment)
│   └── tfsec / checkov (security scan)
├── Merge to main → GitHub Actions: Full Deploy Pipeline
│   ├── terraform apply (hub)
│   ├── terraform apply (aws-spoke)
│   ├── terraform apply (azure-spoke) [via Azure credentials OIDC]
│   ├── terraform apply (gcp-spoke) [via GCP credentials OIDC]
│   └── post-deploy smoke tests
└── Manual (workflow_dispatch) → Destroy Pipeline
    ├── terraform destroy (spokes first)
    └── terraform destroy (hub last)
  
```

GitHub Actions OIDC:

- AWS: [aws-actions/configure-aws-credentials](#) with OIDC role
- Azure: [azure/login](#) with OIDC (federated credentials on service principal)
- GCP: [google-github-actions/auth](#) with Workload Identity Federation

7. Well-Architected Framework Mapping

Pillar	Key Decisions	Toggle in Terraform
Security	WAF, Cognito auth (Azure), GCP direct STS federation, KMS encryption, Secrets Manager, VPC endpoints, least-privilege IAM, public repo secret controls	<code>enable_waf</code> , <code>enable_vpc_endpoints</code>
Reliability	ECS Fargate managed service, Bedrock model fallback, Aurora Serverless auto-pause, KB sync poll-and-wait	<code>bedrock_fallback_model</code> , <code>aurora_auto_pause</code>
Performance	ElastiCache Redis for LiteLLM caching, VPC endpoints to avoid NAT latency, pgvector HNSW index	<code>enable_redis_cache</code>
Cost Optimisation	All serverless/on-demand, Aurora pgvector replaces OpenSearch (~90% cost saving), Infracost integration, destroy runbook	<code>enable_waf</code> , <code>enable_redis_cache</code> , <code>enable_knowledge_base</code>
Operational Excellence	GitHub Actions CI/CD (phased), CloudWatch + optional Grafana dashboards, CloudWatch alarms, Terraform modules	All toggleable
Sustainability	Minimal compute footprint, serverless-first, destroy when idle, scale-to-zero Aurora	Core design

8. Key Architecture Decisions Summary

Full rationale in [docs/04_adr_technology_decisions.md](#)

Decision	Choice	Alternatives Considered
AI Gateway	LiteLLM Proxy (self-hosted ECS Fargate)	LiteLLM Cloud, Amazon Bedrock Converse API direct, custom proxy
Hub cloud	AWS	Azure, GCP
Connectivity	Public HTTPS (API GW + WAF)	PrivateLink, Transit Gateway
Auth (Azure)	API Key (stored in Azure Key Vault)	mTLS client certificates
Auth (GCP)	GCP Workload Identity OIDC → AWS STS → IAM Sig V4	Cognito OIDC federation (original design, replaced)
Auth (AWS spoke)	IAM Signature V4 (cross-account STS)	Long-lived access keys
Vector store	Bedrock KB + Aurora Serverless pgvector	OpenSearch Serverless (production option), Pinecone, Weaviate
Observability	CloudWatch (primary) + OTel → Grafana Cloud (optional)	CloudWatch only, Datadog

Decision	Choice	Alternatives Considered
Terraform state	Terraform Cloud (free)	S3 + DynamoDB
Sample app framework	Python FastAPI + Streamlit	Node.js, Go
LiteLLM image	Pinned -stable semver tag + cosign verification	main-latest (rejected — breaking changes)

9. Deployment Architecture Per Phase

Phase	What Gets Deployed	Estimated Monthly Cost
Phase 1: Hub Foundation	VPC, API GW, WAF, Cognito, ECS (LiteLLM), Aurora, Redis, Secrets Mgr	~\$45/month
Phase 1b: Basic CI/CD	GitHub Actions pipelines (plan/apply/secret scan)	\$0
Phase 2: Bedrock Inference	LiteLLM → Bedrock routing, IAM policies	~\$5 (Bedrock usage)
Phase 3: RAG Pipeline	S3 (KB), Bedrock KB, Aurora pgvector schema	+~\$10–15/month
Phase 4: Bedrock Agent	Agent + Lambda (search_knowledge_base)	+~\$2/month
Phase 5–7: Spokes	3x minimal app deployments	+~\$5–15/month
Phase 8: Observability	CloudWatch Dashboards + OTel Collector (optional Grafana)	+~\$3–5/month
Phase 9: External Models	OpenAI + Vertex AI routes, fallback chain	Usage-based
Phase 10: Guardrails	Bedrock Guardrails + LiteLLM callbacks	+~\$2/month
Phase 11: Full CI/CD	Infracost, tfsec, all-workspace pipelines	\$0
Phase 12: Docs + Cost	Infracost config, runbooks, demo script	\$0
Full PoC (all phases running)		~\$90–110/month peak
Idle (hub running, KB schema dropped)		~\$25–30/month
Fully destroyed		\$0

Note: Peak cost reduced from ~\$250/month (v1.0) to ~\$90–110/month due to Aurora pgvector replacing OpenSearch Serverless.